



《人工智能与Python程序设计》——高级循环神经网络与文本生成



人工智能与Python程序设计 教研组

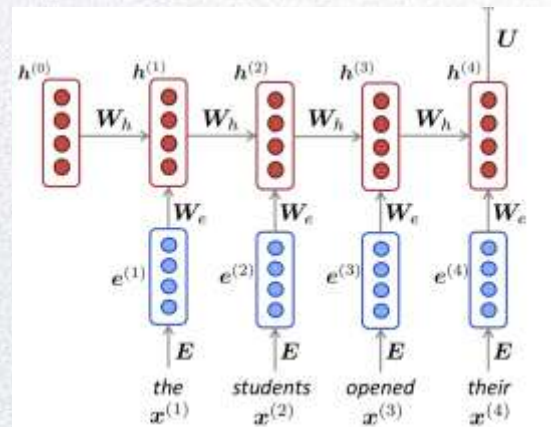
复习



- 上节课学习内容
 - 循环神经网络 (RNN)
 - 建模序列
 - 使用pyTorch实现RNN
 - 基于字符RNN的文本分类器
 - 将字符串 (文本) 表示为RNN可以接受的输入tensor
 - 字符串映射到整数
 - 使用nn.Embedding层将整数映射为多维浮点数向量

回顾：基于循环神经网络的语言模型

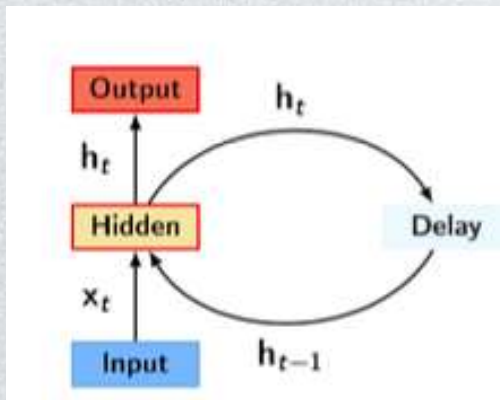
- 基于循环神经网络的语言模型 (Recurrent Neural Networks, RNN)
 - RNN语言模型分析
 - 优点
 - 可以处理任意长度的输入
 - 当前时间的计算可以利用之前的信息
 - 模型大小并不随输入的长度变化



循环神经网络

- 循环神经网络的结构

- 具有一组隐状态 (hidden state) 向量: h_t
- 隐状态向量 h_t 由当前输入 x_t 和前一个隐状态向量决定 h_{t-1}
- 在给定一个输入序列 $X = (x_1, x_2, \dots, x_t, \dots, x_n)$, “循环的” 更新 h_t



$$h_t = \begin{cases} 0 & \text{if } t = 0 \\ f(h_{t-1}, x_t) & \text{otherwise} \end{cases}$$



循环神经网络

- 可以定义不同的更新隐状态向量的方法
- 一种简单的方法是：

$$h_t = f(h_{t-1} \cdot U + x_t \cdot W + b)$$

- 其中：

变量	含义	维度(Shape)
h_t	t 时刻隐状态向量	(1, d)
x_t	t 时刻输入向量	(1, k)
f	非线性映射	-
U	隐状态转移权重矩阵	(d, d)
W	输入到隐状态映射矩阵	(k, d)
b	隐状态向量偏置	(1, d)



使用pyTorch实现循环神经网络

- 使用pyTorch实现该函数:

$$h_t = f(h_{t-1} \cdot U + x_t \cdot W + b)$$

```
class RNNCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()

        self.hidden_size = hidden_size
        normalization_factor = hidden_size ** 0.5
        self.W = nn.Parameter(torch.randn(input_size, hidden_size) / normalization_factor)
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)
        self.non_linear = nn.ReLU()

    def forward(self, input, hidden=None):
        if hidden is None:
            hidden = torch.zeros(1, self.hidden_size)
        hidden = hidden.matmul(self.U) + input.matmul(self.W) + self.b
        hidden = self.non_linear(hidden)
        return hidden
```




使用pyTorch实现循环神经网络

- 通过循环调用该函数，可以处理变长序列

```
rnn_cell = RNNCell(20, 10)
input_tensor = torch.randn(2, 1, 20)
hidden = None
for i in range(input_tensor.shape[0]):
    hidden = rnn_cell(input_tensor[i], hidden)
    print(i, hidden)
```

```
0 tensor([[0.7448, 1.4827, 3.0089, 3.0465,
           1.0028]], grad_fn=<ReluBackward0>)
1 tensor([[0.0000, 1.1701, 0.0000, 0.0000,
           0.0000]], grad_fn=<ReluBackward0>)
```

```
input_tensor = torch.randn(5, 1, 20)
hidden = None
for i in range(input_tensor.shape[0]):
    hidden = rnn_cell(input_tensor[i], hidden)
    print(i, hidden)
```

```
0 tensor([[0.0000, 0.0000, 0.4602, 1.6214, 1.8450, 0.0000, 1.2190, 0.0000, 0.3957,
           0.0000]], grad_fn=<ReluBackward0>)
1 tensor([[1.0400, 1.8433, 3.1282, 0.0000, 1.5801, 0.1586, 0.3469, 0.3237, 3.1358,
           0.0000]], grad_fn=<ReluBackward0>)
2 tensor([[0.0000, 0.6067, 0.0000, 0.0000, 0.3844, 0.2232, 0.0000, 0.0000, 2.5534,
           0.1269]], grad_fn=<ReluBackward0>)
3 tensor([[0.0000, 0.0000, 2.3848, 0.0000, 0.1908, 0.1591, 0.0000, 1.5654, 0.0000,
           0.0000]], grad_fn=<ReluBackward0>)
4 tensor([[1.1500, 1.8459, 1.8786, 1.2277, 1.3489, 0.0000, 0.0000, 0.0000, 0.0000,
           0.8068]], grad_fn=<ReluBackward0>)
```



基于字符级别循环神经网络的文本分类模型

- 使用pyTorch实现一个基于RNN的文本分类器
 - 输入：人名
 - 输出：这是哪种语言中常见的人名

数据准备

- 数据标注
- 训练集/验证集/测试集分割
- 特征提取

模型训练

- 分类损失函数
- 损失函数优化和参数调优

模型测试

- 性能评价



● 基于字符级别循环神经网络的文本分类模型

- 数据准备：
 - 如何表示文本输入？
- 模型构建
 - RNN模型构建
 - 循环使用RNN模型得到预测输出(output)
 - 定义损失函数
- 模型训练
 - 双重循环
 - 外层循环遍历数据集，内层循环每次选择一个数据计算梯度更新模型参数
- 模型测试
 - 使用topk函数，输出output向量中值最大的元素的位置

如何表示文本输入？

- 第1步：构建一个字典，将每个字符映射为一个整数，组成一个tensor
 - 首先使用**集合**，得到所有在人名中出现的字符，保存在一个**有序列表**中
 - 然后构建**字典**，将每个**字符**映射为这个字符在有序列表中出现的位置（**整数**）
 - 整数的取值范围为0~V-1，V是字符集合的大小
 - 最后使用该字典，将人名字符串，转变为一个**保存了整数的tensor**
 - 注意：name tensor long为一个shape为(字符串长度,1)的列向量，其中的元素为torch.long类型的整数，取值范围为0~V-1

```
[11]: # 将一个字符映射为一个整数tensor
import torch
index2char = sorted(set(''.join(names)))
char2index = {c:i for i, c in enumerate(index2char)}
name_tensor_long = torch.tensor([[char2index[c]] for c in 'Alice'], dtype=torch.long)

print('index2char', index2char[0:10])
print(char2index)
print(name_tensor_long)

index2char [' ', "'", 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
{' ': 0, "'": 1, 'A': 2, 'B': 3, 'C': 4, 'D': 5, 'E': 6, 'F': 7, 'G': 8, 'H': 9, 'I': 10, 'J': 11, 'K': 12, 'L': 13, 'M': 14, 'N': 15, 'O': 16, 'P': 17, 'Q': 18, 'R': 19, 'S': 20, 'T': 21, 'U': 22, 'V': 23, 'W': 24, 'X': 25, 'Y': 26, 'Z': 27, 'a': 28, 'b': 29, 'c': 30, 'd': 31, 'e': 32, 'f': 33, 'g': 34, 'h': 35, 'i': 36, 'j': 37, 'k': 38, 'l': 39, 'm': 40, 'n': 41, 'o': 42, 'p': 43, 'q': 44, 'r': 45, 's': 46, 't': 47, 'u': 48, 'v': 49, 'w': 50, 'x': 51, 'y': 52, 'z': 53}
tensor([[ 2],
        [39],
        [36],
        [30],
        [32]])
```



如何表示文本输入？

- 第2步：使用nn.Embedding层，将一个整数映射为一个多维浮点数组成的向量，使其能作为RNN模型的输入
 - 这种方法常被成为嵌入表示
 - 嵌入维度通常为几十维~几百维

```
[15]: # 使用nn.Embedding层，将用0~V-1整数表示词的文本转化成词嵌入表示矩阵
      from torch import nn
      embedding_dimension = 100
      embedding_layer = nn.Embedding(len(char2index), embedding_dimension)
      name_tensor_float = embedding_layer(name_tensor_long)
      print(name_tensor_float.shape)
      name_tensor_float

      torch.Size([5, 1, 100])

[15]: tensor([[[[-2.0978e+00,  1.7269e+00, -5.1127e-01, -2.0617e-01,  9.7511e-01,
          2.0260e+00,  1.1956e+00, -5.6362e-01, -8.4520e-01, -5.3024e-01,
          -5.2115e-01, -1.2494e+00, -2.7861e+00,  5.7239e-01,  6.2490e-01,
          -4.0992e-01,  1.2377e-01, -1.5592e-01,  1.9939e+00, -4.1127e-02,
          -4.1915e-01,  9.6891e-01,  7.7349e-01, -1.4265e-01, -1.7592e-01,
          2.5107e-01, -2.6341e-01, -1.0218e+00,  7.1431e-01,  8.2967e-01,
          -1.3828e+00, -3.6081e-01,  3.5180e+00, -5.4527e-02, -6.9404e-01,
          1.5808e+00, -2.9401e-01, -5.9635e-01,  4.7792e-01,  9.1426e-01,
          -7.4908e-01,  1.0147e+00, -1.0432e+00, -5.8009e-01,  1.2573e+00,
```




RNN模型构建

- 输入：
 - 使用nn.Embedding层将字符映射为嵌入向量

```
# 定义RNN
class RNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size):
        super(RNN, self).__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        normalization_factor = hidden_size ** 0.5
        self.W = nn.Parameter(torch.randn(embedding_size, hidden_size) / normalization_factor)
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)
        self.non_linear = nn.ReLU()
        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input, hidden=None):
        input_embeddings = self.embeddings(input)
        if hidden is None:
            hidden = torch.zeros(1, self.hidden_size)
        hidden = hidden.matmul(self.U) + input_embeddings.matmul(self.W) + self.b
        hidden = self.non_linear(hidden)
        output = self.output(hidden)
        return output, hidden
```

RNN模型构建

- 计算隐状态向量:
 - 按照公式 $h_t = f(h_{t-1} \cdot U + x_t \cdot W + b)$, 计算隐状态向量
 - RNN的输入 x_t 为embedding层的输出

```
# 定义RNN
class RNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size):
        super(RNN, self).__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        normalization_factor = hidden_size ** 0.5
        self.W = nn.Parameter(torch.randn(embedding_size, hidden_size) / normalization_factor)
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)
        self.non_linear = nn.ReLU()
        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input, hidden=None):
        input_embeddings = self.embeddings(input)
        if hidden is None:
            hidden = torch.zeros(1, self.hidden_size)
        hidden = hidden.matmul(self.U) + input_embeddings.matmul(self.W) + self.b
        hidden = self.non_linear(hidden)
        output = self.output(hidden)
        return output, hidden
```



RNN模型构建

- 输出：
 - 使用线性层得到一个维数和类别数量 (output_size=18)相同的向量

```
# 定义RNN
class RNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size):
        super(RNN, self).__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        normalization_factor = hidden_size ** 0.5
        self.W = nn.Parameter(torch.randn(embedding_size, hidden_size) / normalization_factor)
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)
        self.non_linear = nn.ReLU()
        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input, hidden=None):
        input_embeddings = self.embeddings(input)
        if hidden is None:
            hidden = torch.zeros(1, self.hidden_size)
        hidden = hidden.matmul(self.U) + input_embeddings.matmul(self.W) + self.b
        hidden = self.non_linear(hidden)
        output = self.output(hidden)
        return output, hidden
```




RNN模型训练

- 定义损失函数:

- 多分类任务

- nn.CrossEntropyLoss()

- 注意: 该损失函数同时计算了softmax函数和交叉熵函数

$$\text{loss}(\mathbf{Z}, \mathbf{y}) = - \sum_{i=0}^{N-1} \log\left(\frac{e^{\mathbf{Z}[i, \mathbf{y}[i]]}}{\sum_j e^{\mathbf{Z}[i, j]}}\right)$$

```
# 定义RNN分类模型
class RNNClassifier():
    def __init__(self, char2index, output_dim, embedding_dim=128, hidden_dim=128):
        """
        创建模型和优化器, 初始RNN模型和优化器超参数
        """
        self.lr = 0.001
        self.epoches = 20

        self.num_chars = len(char2index) # 用于将字符映射到整数的字典, 其大小为字典大小
        self.char2index = char2index

        self.rnn = RNN(self.num_chars, embedding_dim, hidden_dim, output_dim)
        self.optimizer = torch.optim.SGD(self.rnn.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```



RNN模型训练

- 训练函数：外层循环
 - 多次 (epoches次) 遍历训练数据
 - 每次遍历前随机打乱数据的顺序

```
def train(self, X, y):  
    .....  
    训练模型  
    输入：  
        X: 训练数据, (N, ), 每个元素是包含名字信息的字符串  
        y: 训练数据的真实分类, (N, 1), 每个元素是名字所属的类别  
    返回：  
        epoch_losses: 每个epoch的平均损失函数值  
    .....  
    epoch_losses = []  
    data_size = X.shape[0]  
    for epoch in range(self.epoches):  
        losses = []  
  
        # 每轮训练前打乱数据的顺序  
        shuffled_index = np.random.permutation(data_size)  
        X, y = X[shuffled_index], y[shuffled_index]  
  
        for i in tqdm(range(data_size)):  
            # x, y = X[i], y[i]
```



RNN模型训练

- 训练函数：内层循环
 - 依次遍历每一个训练数据，计算损失函数，进行反向传播更新模型参数
 - tqdm：可用于显示进度条，迭代速度，预计完成时间等信息

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数，代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    #*循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)
```

```
100%|██████████| 14051/14051 [00:26<00:00, 533.24it/s]
0%|          | 51/14051 [00:00<00:27, 507.03it/s]
```




RNN模型训练

- 训练函数：内层循环
 - 将特征（人名字符串）转化为nn.Embeddings层需要的整数tensor

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    ##循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 循环计算更新隐状态向量和输出向量

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    ##循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 10)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 利用最后一次得到的输出向量output和预测目标y，计算损失函数

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```




RNN模型训练

- 训练函数：内层循环
 - 模型梯度清零

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 调用backward计算梯度

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 利用优化器更新模型参数

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```




RNN模型训练

- 训练函数：内层循环
 - 记录loss，保存到losses列表中

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：外层循环
 - 利用losses列表中记录的loss
 - 计算每次遍历数据集（每个epoch）的平均loss

```
self.optimizer.step()

# 记录loss
losses.append(loss.item())

# 计算每个epoch的平均loss并输出
epoch_losses.append(sum(losses) / data_size)
print("epoch: {}, avg loss is: {}".format(epoch, epoch_losses[-1]))

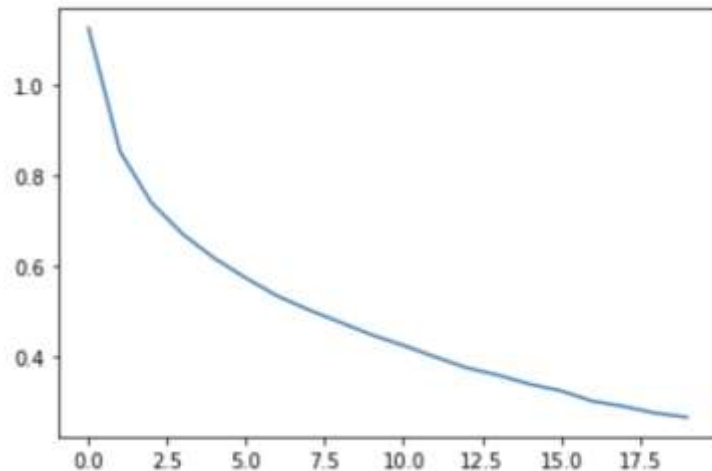
return epoch_losses
```

RNN模型训练

- 损失函数变化曲线：
 - 观察训练过程中损失函数的变化情况是一个调试机器学习和深度学习模型的重要技巧
 - 损失函数下降说明：
 - 模型能够拟合训练数据
 - 模型实现基本正确
 - 模型学习率设置合适

```
# 绘制损失函数变化曲线
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

plt.figure()
plt.plot(losses)
plt.show()
```



模型测试

- 基于训练好的模型进行分类：
 - 循环计算更新隐状态向量和输出向量

```
def test(self, X, y):  
    """  
    用训练好的模型在测试集上进行测试  
    输入:  
        X: 测试数据, (N, ), 每个元素是包含名字信息的字符串  
        y: 测试数据的真实分类, (N, 1), 每个元素是名字所属的类别  
    返回:  
        y_pred: 测试数据的预测分类结果  
        accuracy: 分类准确率  
    """  
    data_size = X.shape[0]  
    y_pred = torch.empty(y.shape).long() # 预测分类与真实分类的形状相同  
    for i in range(data_size):  
        x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)  
  
        # *循环*神经网络!  
        hidden = None  
        for j in range(x.shape[0]):  
            output, hidden = self.rnn(x[j], hidden)  
        # output.shape = (1, 18)  
        top_n, top_i = output.topk(1)  
        y_pred[i] = top_i[0]
```

模型测试

- 基于训练好的模型进行分类：
 - 使用topk函数，找出output中哪一维的值最大，得到模型预测结果

```
def test(self, X, y):  
    """  
    用训练好的模型在测试集上进行测试  
    输入：  
        X: 测试数据, (N, ), 每个元素是包含名字信息的字符串  
        y: 测试数据的真实分类, (N, 1), 每个元素是名字所属的类别  
    返回：  
        y_pred: 测试数据的预测分类结果  
        accuracy: 分类准确率  
    """  
    data_size = X.shape[0]  
    y_pred = torch.empty(y.shape).long() # 预测分类与真实分类的形状相同  
    for i in range(data_size):  
        x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)  
  
        # *循环*神经网络!  
        hidden = None  
        for j in range(x.shape[0]):  
            output, hidden = self.rnn(x[j], hidden)  
  
            # output.shape = (1, 18)  
            top_n, top_i = output.topk(1)  
            y_pred[i] = top_i[0]
```



提纲



人工智能与Python程序设计11-2
高级循环神经网络与文本生成

- ☐ 使用pyTorch实现的循环神经网络
- ☐ 大作业4:基于RNN的古诗词生成
- ☐

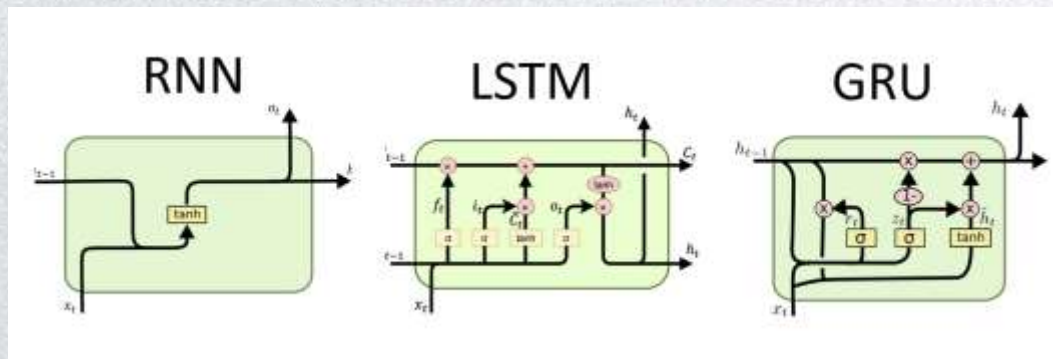


PyTorch自带的RNN模块

- PyTorch自带了三种常见的RNN模型
 - nn.RNN
 - <https://pytorch.org/docs/stable/generated/torch.nn.RNN.html>
 - nn.LSTM
 - <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>
 - nn.GRU
 - <https://pytorch.org/docs/stable/generated/torch.nn.GRU.html>

LSTM vs GRU

- LSTM和GRU是两种被广泛使用的RNN结构
 - 能够解决“梯度消失”问题，适用于建模更长的序列
 - 具体结构请参见附录
- 通常情况下，可以先尝试使用LSTM
- 如果需要提升训练/推理速度或者控制参数数量，可以切换使用GRU





PyTorch自带的RNN模块

- 以RNN模块为例：

```
rnn = nn.RNN(20, 10)
input_tensor = torch.randn(5, 1, 20)
output, last_hidden = rnn(input_tensor)
print('所有隐状态向量: ', output.shape, output)
print('最后一个隐状态向量: ', last_hidden.shape, last_hidden)
```

```
所有隐状态向量: torch.Size([5, 1, 10]) tensor([[[ 0.1059,  0.9938, -0.8119,  0.8183,  0.4405, -0.7251,  0.4852,
          -0.7242,  0.6348,  0.3472]],
         [[-0.2804,  0.5784,  0.0565, -0.9340, -0.2812,  0.6115,  0.7474,
          -0.7019, -0.7243,  0.1096]],
         [[ 0.3313, -0.9363,  0.5748, -0.6004,  0.6984,  0.1894, -0.2603,
          -0.9183, -0.8593,  0.7540]],
         [[-0.5861, -0.5630,  0.9034,  0.5344,  0.1782, -0.1572, -0.2818,
           0.8749,  0.8921, -0.7174]],
         [[ 0.2691, -0.2582,  0.3225, -0.1335,  0.7183,  0.4074, -0.3367,
          -0.0951, -0.0535, -0.6197]]], grad_fn=<StackBackward>)
最后一个隐状态向量: torch.Size([1, 1, 10]) tensor([[[ 0.2691, -0.2582,  0.3225, -0.1335,  0.7183,  0.4074, -0.3367,
          -0.0951, -0.0535, -0.6197]]], grad_fn=<StackBackward>)
```




PyTorch自带的RNN模块

- 构造RNN:
 - `rnn = nn.RNN(input_size, hidden_size)`
 - `embedding_size`: 作为输入的向量维度 (e.g. 128)
 - `hidden_size`: 隐状态向量维度 (e.g. 128)



PyTorch自带的RNN模块

- 调用RNN: `output, last_hidden = rnn(input_tensor)`
 - input:
 - 一个shape为 (序列长度, **batch_size**, embedding_size) 的tensor
 - output:
 - 所有隐状态向量h组成的tensor
 - PyTorch提供的RNN可以在一次调用中计算所有隐状态向量!
 - 一个shape为 (序列长度, **batch_size**, hidden_size) 的tensor
 - last_hidden:
 - 最后一个隐状态向量
 - 一个shape为 (1, **batch_size**, hidden_size) 的tensor



PyTorch自带的RNN模块

- **Batch_size**是什么意思?
- 分批操作 (batching)
 - 在进行一次forward pass时处理batch_size个数据的计算
 - Batch内的计算可以**并行**执行
 - GPU非常擅长并行计算, 通常来说, 增大batch_size能更为充分的利用计算能力

```
[15]: # batch_size = 3的例子
      rnn = nn.RNN(20, 10)
      # 输入为3个长度为5的序列, 序列中每个元素用一个维度为20的embedding向量表示
      input_tensor = torch.randn(5, 3, 20)
      output, last_hidden = rnn(input_tensor)
      print('所有隐状态向量: ', output.shape)
      print('最后一个隐状态向量: ', last_hidden.shape)
```

所有隐状态向量: torch.Size([5, 3, 10])

最后一个隐状态向量: torch.Size([1, 3, 10])



PyTorch自带的RNN模块

- 为什么要同时输出output和last_hidden?
 - output:
 - 隐状态向量h组成的tensor, shape为 (序列长度, batch_size, hidden_size)
 - last_hidden:
 - 最后一个隐状态向量, shape为 (1, batch_size, hidden_size)
- 另一种调用RNN的方法:
 - 循环的依次输入input, 立即得到当前位置的隐状态向量

```
1: # 另一种使用RNN的方法
   rnn = nn.RNN(20, 10)
   input_tensor = torch.randn(5, 1, 20)
   hidden = None
   for i in range(input_tensor.shape[0]):
       # 每次输入x_t和上一轮得到的h_{t-1}
       output, hidden = rnn(input_tensor[i:i+1], hidden)
       print(i, output)

0 tensor([[[ 0.8073, -0.2505,  0.3777, -0.0106,  0.1801, -0.8168, -0.6878,
            -0.1342, -0.7300,  0.9174]]], grad_fn=<StackBackward>)
1 tensor([[[[-0.0535,  0.7281,  0.6237, -0.0785, -0.7233,  0.3636,  0.8399,
             0.2135, -0.4841, -0.5292]]], grad_fn=<StackBackward>)
```

PyTorch自带的RNN模块

- 使用LSTM的例子:

```
# 使用LSTM的例子:  
rnn = nn.LSTM(20, 10)  
input_tensor = torch.randn(5, 1, 20)  
output, last_hidden = rnn(input_tensor)  
print('所有隐状态向量: ', output.shape, output)
```

```
所有隐状态向量: torch.Size([5, 1, 10]) tensor([[[ 1.8491e-02,  1.2720e-01,  8.5417e-02, -1.8219e-01, -8.6898e-02,  
          -3.2382e-02, -9.4329e-02,  3.3682e-01,  1.4314e-01, -2.7969e-02]],  
        [[ 2.0315e-01, -1.1563e-01,  1.0995e-01, -8.5067e-02, -2.0898e-01,  
          8.9114e-03, -2.9707e-02,  1.9040e-01,  4.1042e-01, -4.5317e-04]],  
        [[ 5.7538e-02,  6.7047e-03,  9.4086e-02, -3.0243e-01, -3.3249e-01,  
          -1.9185e-01, -6.1283e-02,  1.9415e-01,  5.6256e-01, -1.4536e-02]],  
        [[ 4.7501e-02,  1.4328e-02,  1.8092e-01, -1.5667e-01, -3.0463e-01,  
          -2.6882e-02,  9.7489e-02, -3.4350e-01,  1.5152e-01,  5.1784e-02]],  
        [[-3.6107e-02,  3.4934e-02,  3.7729e-02,  1.1062e-02, -4.3547e-02,  
          6.9988e-02, -1.9483e-01, -6.8625e-02,  1.6745e-01,  1.7587e-01]]],  
grad_fn=<StackBackward>)
```



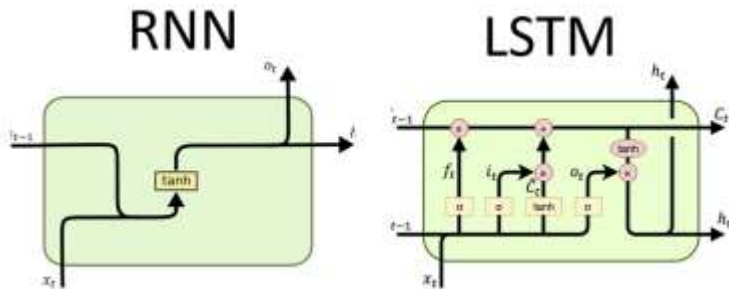
PyTorch自带的RNN模块

- 构造: `rnn = nn.LSTM(embedding_size, hidden_size)`
 - `embedding_size`: 作为输入的词嵌入向量维度 (e.g. 128)
 - `hidden_size`: 隐状态向量位数 (e.g. 128)
- 调用: `output, last_hidden = rnn(input)`
 - `input`:
 - 一个shape为 (序列长度, `batch_size`, `embedding_size`) 的tensor
 - `output`:
 - 隐状态向量h组成的tensor
 - 一个shape为 (序列长度, `batch_size`, `hidden_size`) 的tensor



PyTorch自带的RNN模块

- 调用: `output, last_hidden = rnn(input)`
 - output:
 - 隐状态向量h组成的tensor
 - 一个shape为 (序列长度, batch_size, hidden_size) 的tensor
 - last_hidden:
 - LSTM输出 (`h_t`, `c_t`) , 分别为隐状态向量和长时记忆单元向量组成的tensor, 维度为(1, batch_size, hidden_size)



```
[29]: h, c = last_hidden
      print('h:', h.shape, h)
      print('c:', c.shape, c)

h: torch.Size([1, 1, 10]) tensor([[[ 0.1152, -0.3024, -0.1946,  0.2177, -0.5301,
      -0.2037,  0.1094, -0.1374]]], grad_fn=<StackBackward>)
c: torch.Size([1, 1, 10]) tensor([[[ 0.1774, -0.4394, -0.2535,  0.3199, -0.9555,
      -0.7238,  0.1723, -0.3853]]], grad_fn=<StackBackward>)
```



Pytorch自带的RNN、LSTM、GRU实现

- 使用PyTorch自带的RNN模块实现的字符级别文本分类模型

```
# 定义RNN
class TorchRNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size, model='RNN'):
        super().__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        assert model in ['RNN', 'LSTM', 'GRU'], "Only support RNN/LSTM/GRU model"

        if model == 'RNN':
            self.rnn = nn.RNN(embedding_size, hidden_size)
        elif model == 'LSTM':
            self.rnn = nn.LSTM(embedding_size, hidden_size)
        elif model == 'GRU':
            self.rnn = nn.GRU(embedding_size, hidden_size)

        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input):
        input_embeddings = self.embeddings(input)
        hidden, _ = self.rnn(input_embeddings)
        output = self.output(hidden)
        return output, hidden
```



RNN模型构建

- 输入：
 - 使用nn.Embedding层将字符映射为嵌入向量

```
# 定义RNN
class TorchRNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size, model='RNN'):
        super().__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        assert model in ['RNN', 'LSTM', 'GRU'], "Only support RNN/LSTM/GRU model"

        if model == 'RNN':
            self.rnn = nn.RNN(embedding_size, hidden_size)
        elif model == 'LSTM':
            self.rnn = nn.LSTM(embedding_size, hidden_size)
        elif model == 'GRU':
            self.rnn = nn.GRU(embedding_size, hidden_size)

        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input):
        input_embeddings = self.embeddings(input)
        hidden, _ = self.rnn(input_embeddings)
        output = self.output(hidden)
        return output, hidden
```


RNN模型构建

- 构造模型：
 - 根据参数在RNN/LSTM/GRU中选择一种RNN模型

```
# 定义RNN
class TorchRNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size, model='RNN'):
        super().__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        assert model in ['RNN', 'LSTM', 'GRU'], "Only support RNN/LSTM/GRU model"

        if model == 'RNN':
            self.rnn = nn.RNN(embedding_size, hidden_size)
        elif model == 'LSTM':
            self.rnn = nn.LSTM(embedding_size, hidden_size)
        elif model == 'GRU':
            self.rnn = nn.GRU(embedding_size, hidden_size)

        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input):
        input_embeddings = self.embeddings(input)
        hidden, _ = self.rnn(input_embeddings)
        output = self.output(hidden)
        return output, hidden
```



RNN模型构建

- 计算隐状态向量：
 - 使用nn.RNN/LSTM/GRU一次性计算所有隐状态向量
 - Shape为 (length, batch_size, hidden_size)

```
# 定义RNN
class TorchRNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size, model='RNN'):
        super().__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        assert model in ['RNN', 'LSTM', 'GRU'], "Only support RNN/LSTM/GRU model"

        if model == 'RNN':
            self.rnn = nn.RNN(embedding_size, hidden_size)
        elif model == 'LSTM':
            self.rnn = nn.LSTM(embedding_size, hidden_size)
        elif model == 'GRU':
            self.rnn = nn.GRU(embedding_size, hidden_size)

        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input):
        input_embeddings = self.embeddings(input)
        hidden, _ = self.rnn(input_embeddings)
        output = self.output(hidden)
        return output, hidden
```



RNN模型构建

- 输出：
 - 使用线性层得到将hidden映射为shape为(length, batch_size, output_size)的tensor, 其中output_size=18

```
# 定义RNN
class TorchRNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size, model='RNN'):
        super().__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        assert model in ['RNN', 'LSTM', 'GRU'], "Only support RNN/LSTM/GRU model"

        if model == 'RNN':
            self.rnn = nn.RNN(embedding_size, hidden_size)
        elif model == 'LSTM':
            self.rnn = nn.LSTM(embedding_size, hidden_size)
        elif model == 'GRU':
            self.rnn = nn.GRU(embedding_size, hidden_size)

        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input):
        input_embeddings = self.embeddings(input)
        hidden, _ = self.rnn(input_embeddings)
        output = self.output(hidden)
        return output, hidden
```


RNN模型训练

- 双重循环

- 外层循环遍历数据集，每次遍历前打乱数据集顺序
- 内层循环每次选择一个batch(16个) 的数据计算梯度更新模型参数

```
def train(self, X, y):  
    """  
    训练模型  
    输入:  
        X: 训练数据, (N, ), 每个元素是包含名字信息的字符串  
        y: 训练数据的真实分类, (N, 1), 每个元素是名字所属的类别  
    返回:  
        epoch_losses: 每个epoch的平均损失函数值  
    """  
    epoch_losses = []  
    data_size = X.shape[0]  
    for epoch in range(self.epochs):  
        losses = []  
  
        # 每轮训练前打乱数据的顺序  
        shuffled_index = np.random.permutation(data_size)  
        X, y = X[shuffled_index], y[shuffled_index]  
  
        for i in tqdm(range(0, data_size, self.batch_size)):  
            # x.shape = (length, batch_size)  
            x = self.get_batch_tensor(X[i:i+self.batch_size])  
  
            # y_true.shape = (batch_size, ), y 是一个{0, 1, ..., 17}范围内的整数组成的向量, 代表x属于的分类  
            y_true = torch.tensor(y[i:i+self.batch_size], dtype=torch.long)  
            # 上一步得到的y_true.shape为(batch_size, 1). 将其转化为(batch_size, )  
            y_true = y_true.view(-1)
```



RNN模型训练

- 如何准备一个batch用于训练?
 - 输入: batch_size个字符串
 - 返回: shape为 (max_length, batch_size)的整数tensor, 可用作RNN模型的输入
 - max_length为batch_size个字符串的长度的最大值

```
def get_batch_tensor(self, X):  
    batch_size = len(X)  
    max_len = max(map(len, X))  
    ret_tensor = torch.zeros(max_len, batch_size, dtype=torch.long)  
    for i in range(batch_size):  
        x = torch.tensor([self.char2index[c] for c in X[i]], dtype=torch.long)  
        ret_tensor[-len(X[i]):, i:i+1] = x  
    return ret_tensor
```



RNN模型训练

- 如何准备一个batch用于训练?
 - 返回: shape为 (max_length, batch_size)的整数tensor
 - max_length为batch_size个字符串的长度的最大值
 - Padding操作:
 - 准备一个全0tensor
 - 生成代表字符串的整数 (列) 向量按照末位对齐的方式填入全0tensor中

```
def get_batch_tensor(self, X):  
    batch_size = len(X)  
    max_len = max(map(len, X))  
    ret_tensor = torch.zeros(max_len, batch_size, dtype=torch.long)  
    for i in range(batch_size):  
        x = torch.tensor([self.char2index[c] for c in X[i]], dtype=torch.long)  
        ret_tensor[-len(X[i]):, i:i+1] = x  
    return ret_tensor
```




RNN模型训练

- 内层循环：
 - 调用RNN模型计算出所有输出向量

```
for i in tqdm(range(0, data_size, self.batch_size)):
    # x.shape = (length, batch_size)
    x = self.get_batch_tensor(X[i:i+self.batch_size])

    # y_true.shape = (batch_size, ), y 是一个{0, 1, ..., 17}范围内的整数组成的向量, 代表x属于的分类
    y_true = torch.tensor(y[i:i+self.batch_size], dtype=torch.long)
    # 上一步得到的y_true.shape为(batch_size, 1), 将其转化为(batch_size, )
    y_true = y_true.view(-1)

    output, hidden = self.model(x)
    # output[-1].shape = (batch_size, 18)
    loss = self.loss_function(output[-1], y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item() * self.batch_size)
```



RNN模型训练

- 内层循环：
 - 取出最后一个输出向量和预测目标y_true计算CrossEntropyLoss

```
for i in tqdm(range(0, data_size, self.batch_size)):
    # x.shape = (length, batch_size)
    x = self.get_batch_tensor(X[i:i+self.batch_size])

    # y_true.shape = (batch_size, ), y 是一个{0, 1, ..., 17}范围内的整数组成的向量, 代表x属于的分类
    y_true = torch.tensor(y[i:i+self.batch_size], dtype=torch.long)
    # 上一步得到的y_true.shape为(batch_size, 1), 将其转化为(batch_size, )
    y_true = y_true.view(-1)

    output, hidden = self.model(x)
    # output[-1].shape = (batch_size, 18)
    loss = self.loss_function(output[-1], y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item() * self.batch_size)
```

RNN模型训练

- 内层循环：
 - 取出最后一个输出向量和预测目标y_true计算CrossEntropyLoss

```
for i in tqdm(range(0, data_size, self.batch_size)):
    # x.shape = (length, batch_size)
    x = self.get_batch_tensor(X[i:i+self.batch_size])

    # y_true.shape = (batch_size, ), y 是一个{0, 1, ..., 17}范围内的整数组成的向量, 代表x属于的分类
    y_true = torch.tensor(y[i:i+self.batch_size], dtype=torch.long)
    # 上一步得到的y_true.shape为(batch_size, 1), 将其转化为(batch_size, )
    y_true = y_true.view(-1)

    output, hidden = self.model(x)
    # output[-1].shape = (batch_size, 18)
    loss = self.loss_function(output[-1], y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item() * self.batch_size)
```


RNN模型训练

- 内层循环：
 - 反向传播计算梯度，利用优化器更新模型参数

```
for i in tqdm(range(0, data_size, self.batch_size)):
    # x.shape = (length, batch_size)
    x = self.get_batch_tensor(X[i:i+self.batch_size])

    # y_true.shape = (batch_size, ), y 是一个{0, 1, ..., 17}范围内的整数组成的向量，代表x属于的分类
    y_true = torch.tensor(y[i:i+self.batch_size], dtype=torch.long)
    # 上一步得到的y_true.shape为(batch_size, 1)，将其转化为(batch_size, )
    y_true = y_true.view(-1)

    output, hidden = self.model(x)
    # output[-1].shape = (batch_size, 18)
    loss = self.loss_function(output[-1], y_true)

    # 反向传播，更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item() * self.batch_size)
```

RNN模型训练

- 与上节课实现的RNN进行对比:

```
[28]: classifier = RNNClassifier(char2index, len(country2index))  
      losses = classifier.train(X_train, y_train)
```

```
100%|██████████|  
  0%|  
epoch: 0, avg loss  
100%|██████████|  
  0%|  
epoch: 1, avg loss
```

```
[32]: classifier = RNNClassifier(char2index, len(country2index), model='RNN')  
      rnn_losses = classifier.train(X_train, y_train)
```

```
100%|██████████| 879/879 [00:01<00:00, 573.72it/s]  
  7%|███| 58/879 [00:00<00:01, 573.54it/s]  
epoch: 0, avg loss is: 1.2575088770701262  
100%|██████████| 879/879 [00:01<00:00, 578.64it/s]  
  7%|███| 60/879 [00:00<00:01, 594.21it/s]  
epoch: 1, avg loss is: 0.9567025708571913
```

- 训练速度明显提升，遍历数据集一次所花的时间大大缩短
 - 使用较大的batch_size通常能提升计算效率



总结

- PyTorch的nn工具包中提供了三种常见RNN的实现：
 - RNN/LSTM/GRU
- 如何使用PyTorch提供的RNN：
 - 如何构造模型？
 - 模型的输入和输出的定义和形状？
 - `output, hidden = rnn(input)`
 - 一次性计算所有隐状态向量vs用循环依次得到隐状态向量
- 使用PyTorch提供的RNN/LSTM/GRU实现基于RNN的语言模型和语言生成模型
 - 如何准备一个batch的输入？
 - 一次性输入一个batch的优点是什么？



提纲



人工智能与Python程序设计11-2
高级循环神经网络与文本生成

- ☐ 使用PyTorch实现的循环神经网络
- ☐ 大作业4:基于RNN的古诗词生成
-



基于RNN的古诗词生成

- 实现一个基于RNN的古诗词生成模型
- 具体要求：
 - 模型构建：继承nn.Module，实现PoetryModel类
 - 模型训练：基于古诗词数据集，完成PoetryModel的训练
 - 模型测试：分别实现test和acrostic_test函数，完成诗歌补完和藏头诗生成
- 提示：
 - 在完整数据上训练模型耗时较长
 - 使用一部分数据（如前1000首诗歌）进行调试



回顾：机器学习的基本流程

数据准备

- 数据标注
- 训练集/验证集/测试集分割
- 特征提取



模型训练

- 分类损失函数
- 损失函数优化和参数调优



模型测试

- 性能评价指标
- 交叉验证

数据准备



- 对古诗词进行预处理：
 - 去除作者、标题、编号等信息
 - 将古诗词转化为为整数tensor
 - 构建ix2word、word2ix映射
 - 利用word2ix映射将古诗词字符串表示为整数tensor
 - 在古诗词开头加入特殊字符<START>
 - 在古诗词结尾加入特殊字符<EOP> (End of Poem)
 - 对长度不同的古诗词进行padding操作，在古诗词开头添加特殊字符</s> (空格)，让每首诗的长度均为max_length (125)
 - 将padding后数据 (pad_data)、ix2word、word2ix保存到文件 Poetry_data_word2ix_ix2word中
 - 上述过程已在 “古诗生成-Step0-数据预处理.ipynb” 实现



数据准备

- 可以使用load_data函数读取处理好的古诗词数据

```
def load_data(data_path):  
    """  
    return word2ix: dict, 每个字对应的序号, 形如u'月' -> 100  
    return ix2word: dict, 每个序号对应的字, 形如'100' -> u'月'  
    return poet_data: numpy数组, 每一行是一首诗对应的字的下标  
    """  
    if os.path.isfile(data_path):  
        data = np.load(data_path, allow_pickle=True)  
        poet_data, word2ix, ix2word = data['data'], data['word2ix'].item(), data['ix2word'].item()  
  
        return word2ix, ix2word, poet_data  
    else:  
        print('[ERROR] Data File Not Exists')  
        exit()
```

```
def main():  
    # 获取数据  
    data_path = './data/Poetry_data_word2ix_ix2word.npz'  
    word2ix, ix2word, poet_data = load_data(data_path)
```



数据准备

- 可以使用decode_poetry函数，解码poet_data tensor中第idx首古诗词
 - 从311,823首古诗词中选择一首
 - 用ix2word将整数映射回字符，并去除三种特殊字符

```
173 def decode_poetry(idx, word2ix, ix2word, poet_data):
174     """
175     解码诗歌数据
176     输入:
177         idx: 第几首诗歌(共311823首, idx in [0, 311822])
178     """
179     assert(idx < poet_data.shape[0] and idx >= 0 )
180
181     row = poet_data[idx]
182
183     results = ''.join([
184         ix2word[char] if ix2word[char] != '</s>' and ix2word[char] != '<EOP>'
185         and ix2word[char] != '<START>' else ''
186         for char in row
187     ])
188     return results
```




数据准备

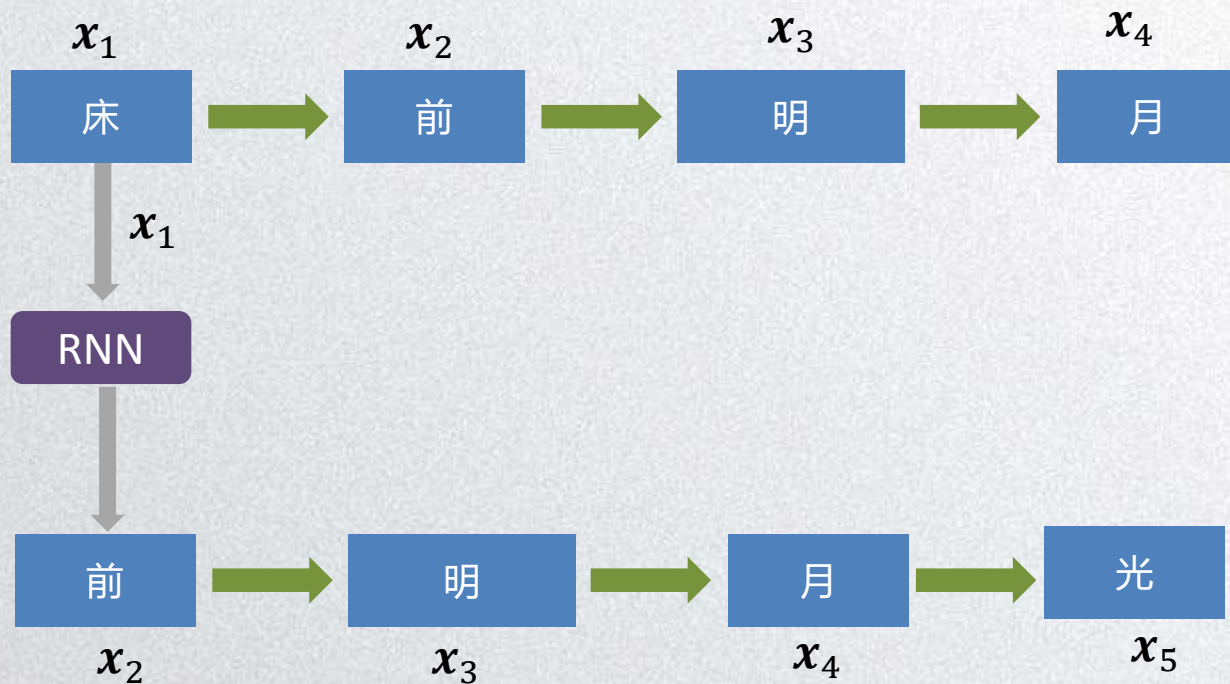
- 使用poet_data构建dataloader
 - 遍历该dataloader可以一次取出batch_size个训练数据
 - 遍历得到的data_的shape为 (batch_size, max_length)
 - 将其转置, 使其shape为(max_length, batch_size)
 - 调试时选取前1000首诗歌, 正式训练时可增大数据规模

```
201 # 转换为tensor与dataloader
202 poet_data = poet_data[:1000, ] # 为测试方便, 只截取了前1000条数据训练。后续代码跑通了并且可以用GPU时, 可以用全部数据
203 poet_data = torch.from_numpy(poet_data)
204 dataloader = data.DataLoader(poet_data,
205                               batch_size=128,
206                               shuffle=True)
```

```
for epoch in range(self.epochs):
    loss_list = []
    for ii, data_ in enumerate(dataloader):
        print('epoch {}: batch {}'.format(epoch, ii), end='\r')
        # 训练
        data_ = data_.long().transpose(1, 0).contiguous().to(self.device)
```

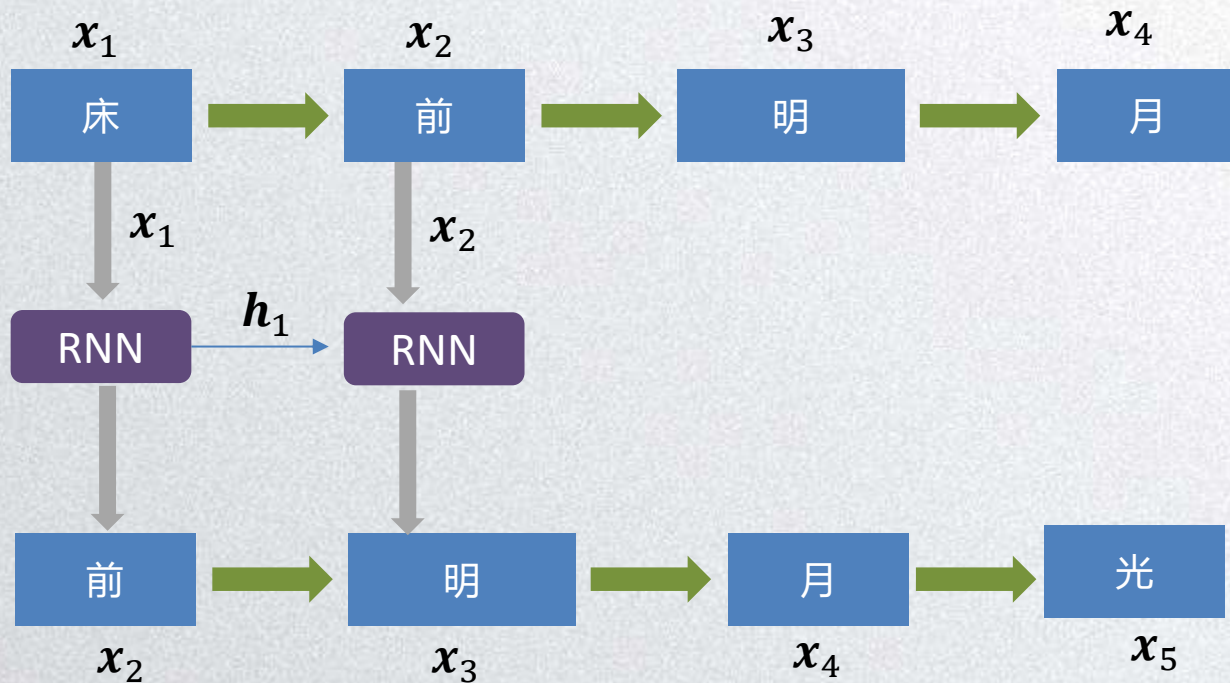
模型构建

- 使用RNN进行诗歌生成
 - 基于0~t-1个字，预测第t个字



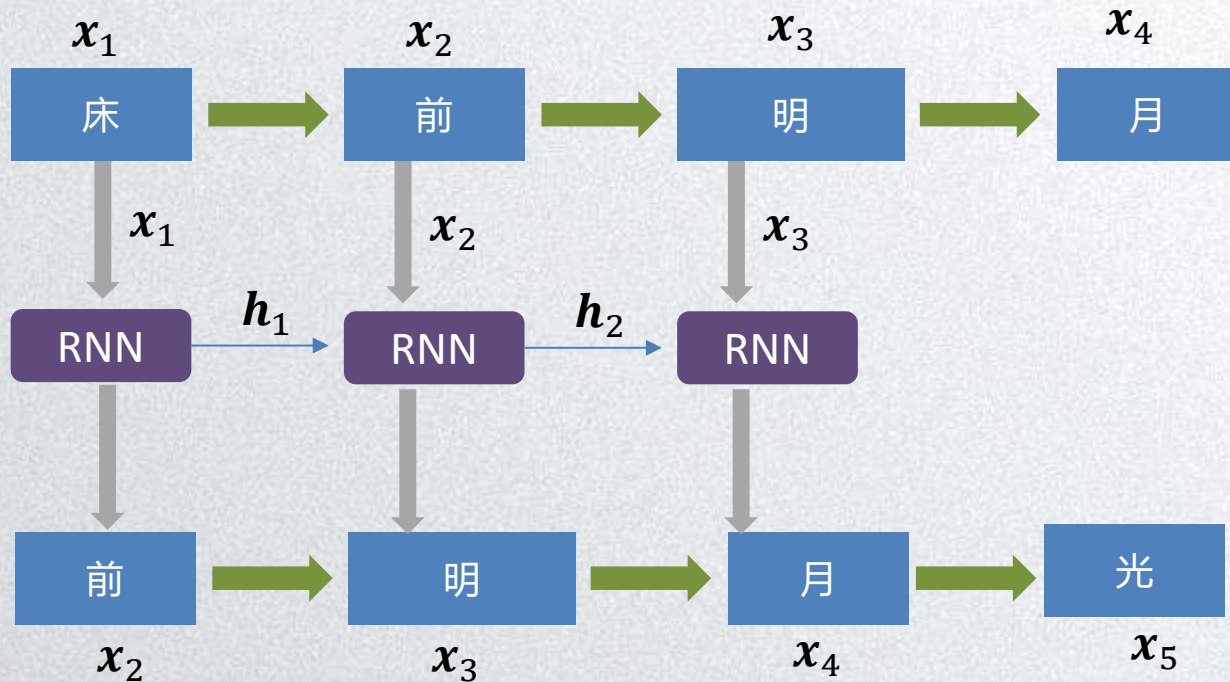
模型构建

- 使用RNN进行诗歌生成
 - 基于0~t-1个字，预测第t个字



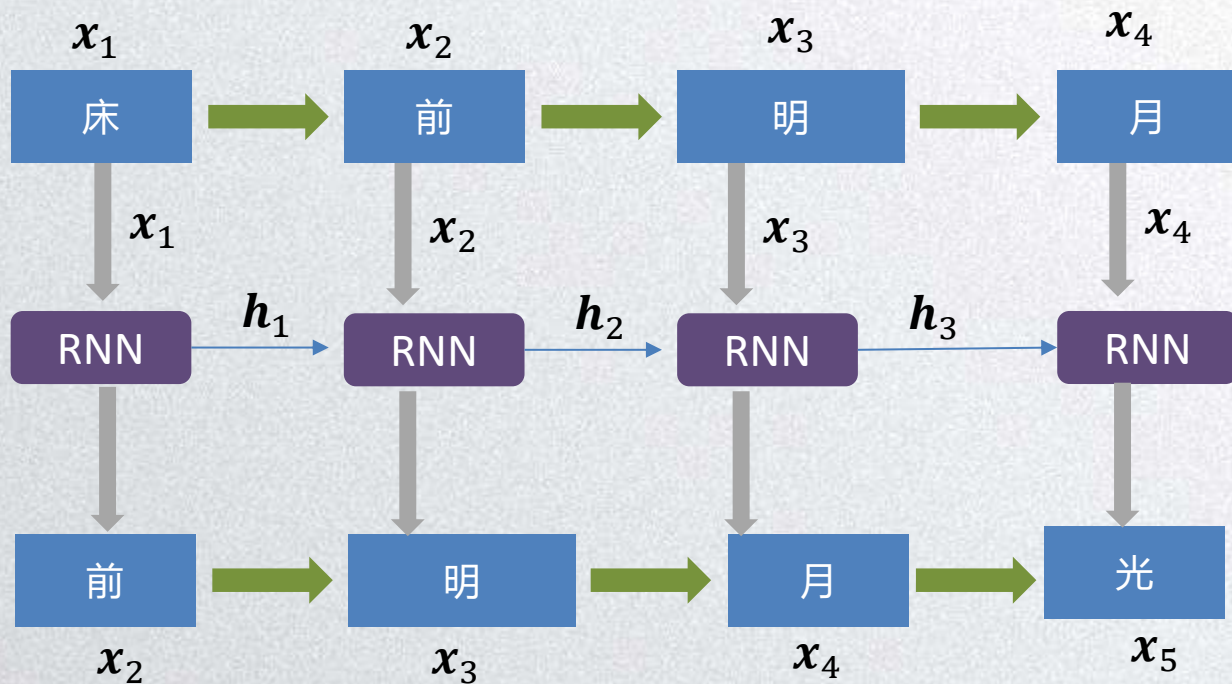
模型构建

- 使用RNN进行诗歌生成
 - 基于0~t-1个字，预测第t个字



模型构建

- 使用RNN进行诗歌生成
 - 基于0~t-1个字，预测第t个字





模型构建

- 使用RNN进行诗歌生成
 - 基于0~t-1个字，预测第t个字
 - RNN的输入(x): `data_[:-1, :]`, 前max_length-1个字
 - 输入: “床前明月”
 - RNN的输出目标(y): `data_[1:, :]`, 后max_length-1个字
 - 预测: “前明月光”

```
input_, target = data_[:-1, :], data_[1:, :]  
output, _ = self.model(input_)
```

- 预测是一个多分类任务: 分类数=字典大小 (`len(ix2word)`)
 - 使用`nn.CrossEntropyLoss`
 - 注意output和target的shape是否匹配! 详见train函数注释

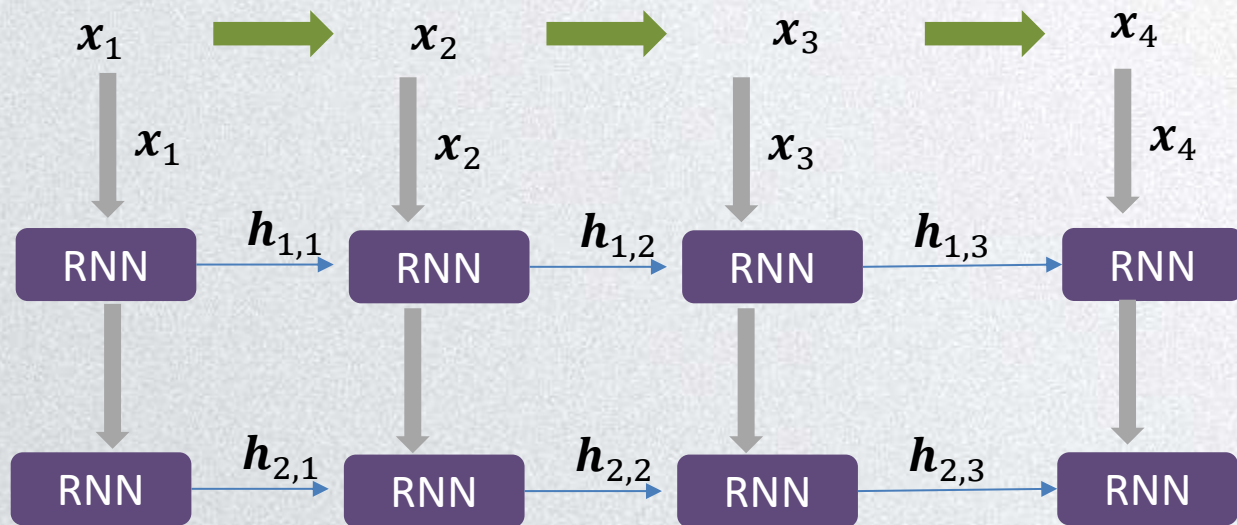
模型构建

- 实现PoetryModel:
 - `__init__`
 - `forward`
- 推荐使用2层LSTM模型

```
17 class PoetryModel(nn.Module):
18     def __init__(self, vocab_size, embedding_dim, hidden_dim):
19         """
20         *****请在此写入你的代码*****
21         定义模型:
22         1. 定义模型隐藏层维度, hidden_dim为读入的隐藏层维度
23         2. 使用词嵌入表示(word embedding), embedding_dim为读入的嵌入向量的维度
24         3. 定义LSTM模型 (推荐使用2层LSTM)
25         4. 定义线性模型 (从 hidden_dim 映射到 vocab_size)
26         """
27         pass # 完成该函数后请将这行删除
28
29     def forward(self, input, hidden=None):
30         """
31         *****请在此处输入你的代码*****
32         输入: input, 它的size是(sequence_length, batch_size)
33         输出 (返回值): output(预测值), hidden(隐藏层的值)
34         * output的size是(sequence_length, batch_size, vocab_size)
35         * hidden的size是(4, batch_size, hidden_size)
36         * h_0 (2, batch_size, hidden_size)
37         * c_0 (2, batch_size, hidden_size)
38         定义模型函数:
39         * 判断输入的参数是否有hidden, 没有的话新建一个全0的tensor
40         * 将input进行词向量嵌入
41         * 使用lstm模型
42         * 用线性层将output映射到vocab_size的维度上
43         * 返回output, hidden
44         """
45         pass # 完成该函数后请将这行删除
```

模型构建

- 2层LSTM模型



模型构建

- 2层LSTM模型

```
# 2层LSTM
rnn = nn.LSTM(20, 10, num_layers=2)
input_tensor = torch.randn(5, 1, 20)
output, last_hidden = rnn(input_tensor)
print('所有隐状态向量: ', output.shape, output)

所有隐状态向量: torch.Size([5, 1, 10]) tensor([[-0.8351,  0.8481, -0.8648,  0.8631, -0.1652,  0.0288, -0.0089,
          -0.8049, -0.1012,  0.8518]],

          [[-0.8736,  0.8828, -0.8992,  0.8940, -0.2278,  0.8088, -0.8184,
          -0.8080, -0.1558,  0.8877]],

          [[-0.8938,  0.8739, -0.1335,  0.1085, -0.2361,  0.0158,  0.8184,
          0.8528, -0.1957,  0.1142]],

          [[-0.1082,  0.1348, -0.1421,  0.1082, -0.2353,  0.8256,  0.8586,
          0.1193, -0.2349,  0.1298]],

          [[-0.1097,  0.1061, -0.1492,  0.1166, -0.2523,  0.8252,  0.8428,
          0.1452, -0.2388,  0.1362]]], grad_fn=<StackBackward>)
```

- input:
 - 一个shape为 (序列长度, batch_size, embedding_size) 的tensor
- output:
 - 隐状态向量h组成的tensor
 - 一个shape为 (序列长度, batch_size, hidden_size) 的tensor
 - 只包含最上



模型构建

- 2层LSTM模型
 - last_hidden:
 - LSTM输出 (h_t , c_t) , 分别为隐状态向量和长时记忆单元向量组成的 tensor, 维度为(2, batch_size, hidden_size)

```
0]: h, c = last_hidden
print('h:', h.shape, h)
print('c:', c.shape, c)

h: torch.Size([2, 1, 10]) tensor([[[ 0.3351, -0.0147,  0.2379, -0.1731,  0.0610, -0.1140, -0.0565,
    -0.0426,  0.3072, -0.4322]],
  [[-0.1097,  0.1061, -0.1492,  0.1166, -0.2523,  0.0252,  0.0428,
    0.1452, -0.2308,  0.1362]]], grad_fn=<StackBackward>)
c: torch.Size([2, 1, 10]) tensor([[[ 0.5049, -0.0262,  0.4605, -0.5699,  0.1315, -0.3003, -0.1210,
    -0.1126,  0.5947, -0.8325]],
  [[-0.3622,  0.1758, -0.3942,  0.2157, -0.4870,  0.0438,  0.0766,
    0.2669, -0.3494,  0.2627]]], grad_fn=<StackBackward>)
```



模型训练

- 模型训练时间较长，每隔10个epoch将当前模型保存到self.model_save_path路径下

```
loss_list.append(loss.item())

epoch_loss = sum(loss_list)/len(loss_list)
print("[INFO] loss of epoch %s: %s" % (epoch, epoch_loss))

# 保存模型参数
if epoch % 10 == 0:
    self._save_model(epoch)
```

```
61 def _save_model(self, epoch):
62     """
63     保存模型，用于训练时保存指定epoch的模型
64     """
65     print('[INFO] Saving to %s_%s.pth' % (self.model_save_path, epoch))
66     torch.save(self.model.state_dict(), '%s_%s.pth' % (self.model_save_path, epoch))
67
```



模型训练

- 可以使用_load_model函数读取训练了指定epoch的模型

```
def _load_model(self, epoch):  
    """  
    加载模型，用于测试时加载指定epoch的模型  
    """  
    print('[INFO] Loading from %s_%s.pth' % (self.model_save_path, epoch))  
    self.model.load_state_dict(torch.load('%s_%s.pth' % (self.model_save_path, epoch), map_location=self.device))
```

- 模型训练和调试时：
 - 运行一个训练程序
 - 每10个epoch保存一个模型文件
 - 编写和运行一个测试程序
 - 读取不同epoch保存的模型进行测试



模型测试

- 实现test函数
 - 给定start_words中包含的字符串作为一首诗的开头
 - 依次用循环方式调用self.model，生成下一个字加入到结果中
 - 直到生成代表一首诗的结尾<EOP>特殊符号为止

```
def test(self, start_words, ix2word, word2ix, max_gen_len = 200):  
    """  
    description: 给定几个词，根据这几个词接着生成一首完整的诗歌  
    example:  
        start_words: u'深度学习'  
        生成:  
        深度学习书不怪，今朝月下不胜悲。  
        芒砀月殿春光晓，宋玉堂中夜月升。  
        玉徽美，玉股洁。心似镜，澈圆珠，金炉烟额红芙蓉。  
    hint:  
        一个字一个字的生成诗歌  
        将start_words一个字一个字的作为input输入模型  
        第一次输入模型时，hidden为None，之后hidden都是上次的返回结果  
        当生成的字为 '<EOP>' 诗歌结束生成  
    """  
    pass # 完成该函数后请将这行删除
```



模型测试

- 实现 `acrostic_test` 函数，生成藏头诗
 - 给定 `start_words` 中包含的字符串作为一首诗中每句话的开头
 - 依次用循环方式调用 `self.model`，生成下一个字加入到结果中
 - 如果生成了代表句子结尾的标点符号（。！？），则将 `start_words` 中一个词添加在标点符号后，输入 `self.model` 继续生成
 - 直到 `start_words` 中每个字都被添加到诗句的开头

```
def acrostic_test(self, start_words, ix2word, word2ix, max_gen_len=200):  
    """  
    description: 生成藏头诗  
    example:  
        start_words : u'深度学习'  
        生成:  
        深宫新月明，皎皎明月明。  
        度风飘飘颭，照景澄清明。  
        学化不可夺，低心不可怜。  
        习人不顾盼，仰面空踟蹰。  
    hint:  
        与上一个函数类似，但需要特殊处理一下“藏头”  
        一句结束，即生成“。”或“！”时，诗歌的下一个字为读入的“藏头”。  
        此时模型的读入为“藏头”对应的字，其他情况下模型读入的是上次生成的字。  
        当所有“藏头”都生成了一句诗，诗歌生成完毕。  
    """  
    pass # 完成该函数后请将这行删除
```



谢谢！



提纲

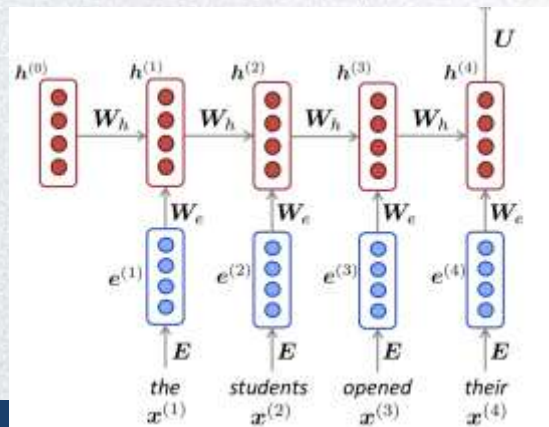
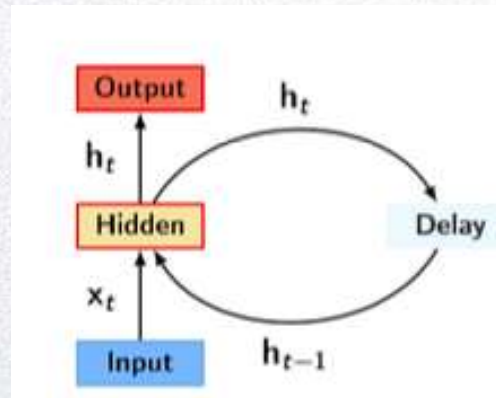


人工智能与Python程序设计11-2
高级循环神经网络与文本生成

- □ 附录：梯度消失和梯度爆炸
- □ 附录：LSTM和GRU的结构
-

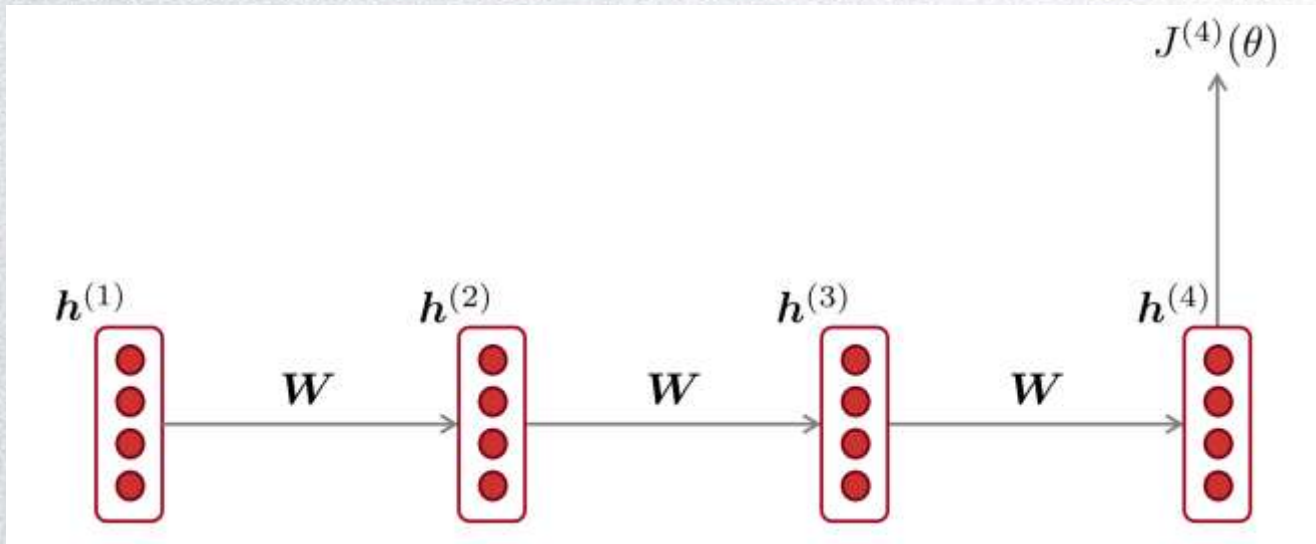
回顾：循环神经网络

- 优点：
 - 可以处理任意长度的输入
 - 当前时间的计算可以利用之前的信息
 - 模型大小并不随输入的长度变化
- 缺点：
 - 循环计算不利于并行计算
 - 训练速度较慢
 - 不能够很好处理长序列
 - 存在信息“遗忘”的情况



回顾：循环神经网络

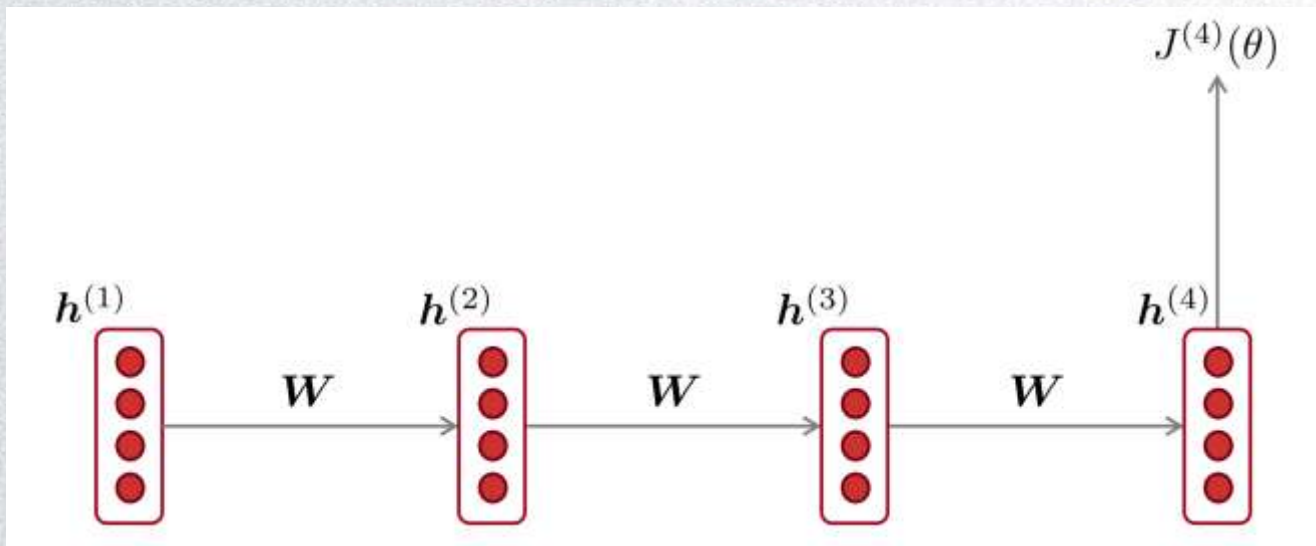
- 循环神经网络的梯度是如何计算的？
 - 通过pyTorch自动求导计算的
 - 不需要手工实现梯度计算，但需要理解原理



回顾：循环神经网络

- 循环神经网络的梯度是如何计算的？

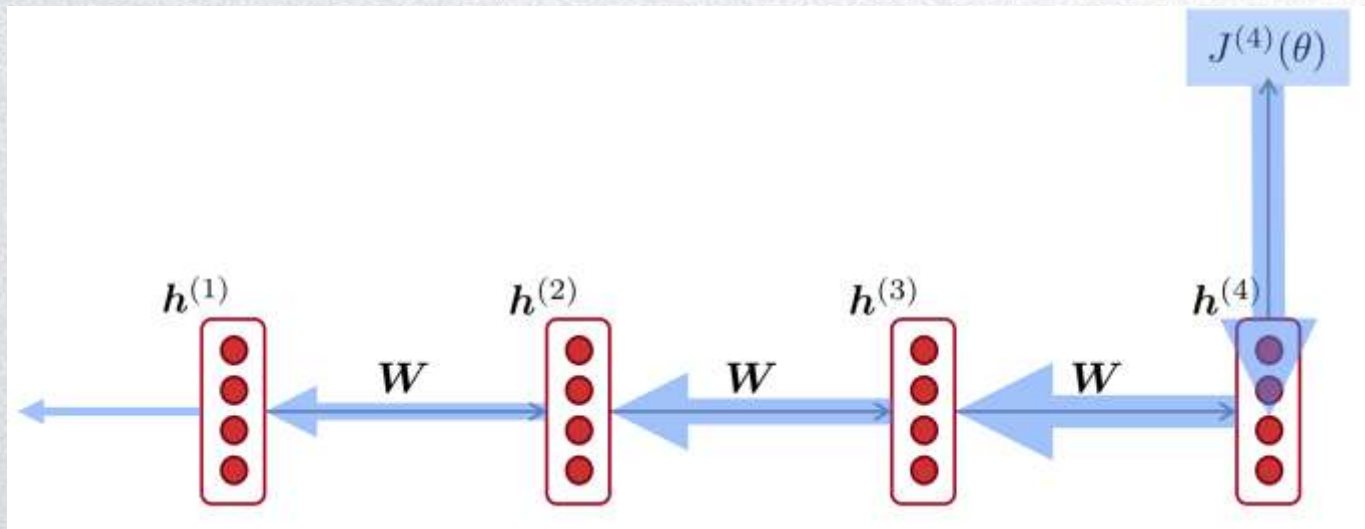
– 例如： $\frac{\partial J^{(4)}}{\partial h^{(1)}} = ?$



回顾：循环神经网络

- 循环神经网络的梯度是如何计算的？

– 链式法则！
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(4)}}{\partial h^{(3)}} \frac{\partial J^{(4)}}{\partial h^{(4)}}$$





回顾：循环神经网络

- 循环神经网络的梯度是如何计算的？

- 链式法则！ $\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(4)}}{\partial h^{(2)}} \frac{\partial J^{(4)}}{\partial h^{(4)}}$

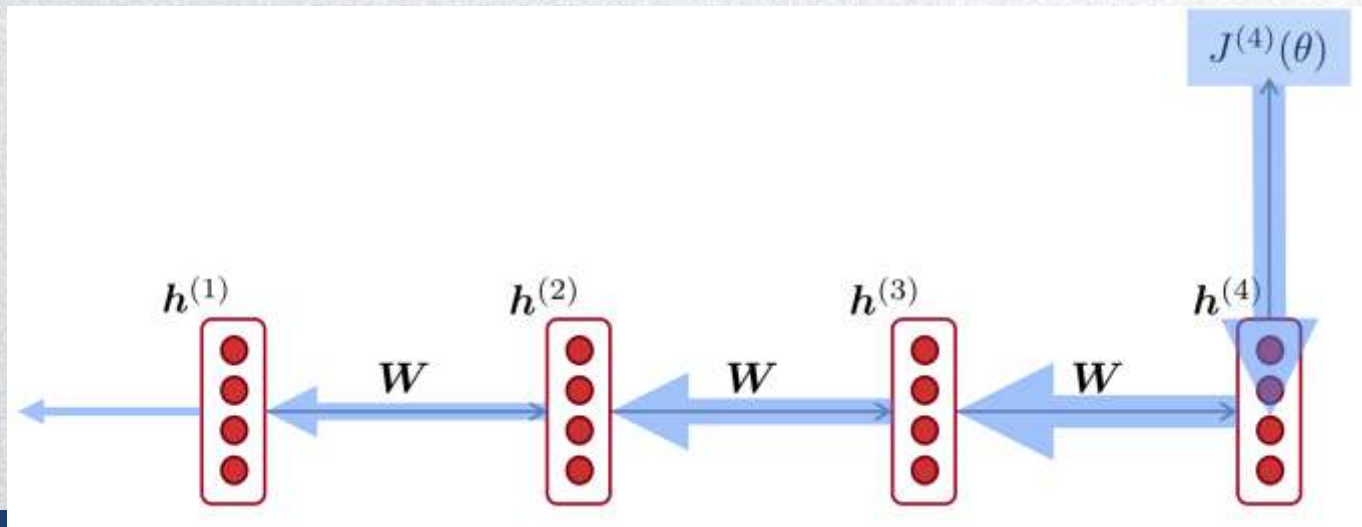
- $h^{(i+1)} = \sigma(W_h h^{(i)} + W_x x^{(i+1)} + b)$

- $\frac{\partial h^{(i+1)}}{\partial h^{(i)}} = \text{diag}\left(\sigma'(W_h h^{(i)} + W_x x^{(i+1)} + b)\right) W_h$

问题：梯度消失和梯度爆炸

- 梯度消失：

- $\frac{\partial h^{(i+1)}}{\partial h^{(i)}}$ 较小：序列长度较长时，无法有效的训练靠前位置，导致靠前位置的输入对预测结果影响很小
- RNN会“遗忘”





问题：梯度消失和梯度爆炸

- 梯度爆炸：

- $\frac{\partial h^{(i+1)}}{\partial h^{(i)}}$ 较大

- 序列长度较长时，每一步更新对参数带来的变化会很大

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$$

- 这会使随机梯度下降无法有效的优化损失函数

- 甚至会导致网络中出现inf（无穷大）或者nan（not-a-number，常在自变量超出函数定义域时出现，如 $\log(-1)=\text{nan}$ ）



提纲



人工智能与Python程序设计11-2
高级循环神经网络与文本生成

- □ 附录：梯度消失和梯度爆炸
- □ 附录：LSTM和GRU的结构
-



高级循环神经网络

- 为了解决梯度消失问题，更复杂的循环神经网络被提出：
 - 长短时记忆网络 (Long Short-Term Memory, LSTM)
 - 门控循环神经网络 (Gated Recurrent Units, GRU)



长短时记忆网络(LSTM)

- 使用隐状态向量 h_t 和长期记忆单元 c_t 来表示当前状态
- 在 t 时刻, LSTM可以选择删除(遗忘)、写入(输入)和读取(输出)长期记忆单元 c_t 的内容

想出了一些方式来解决导数消失的问题

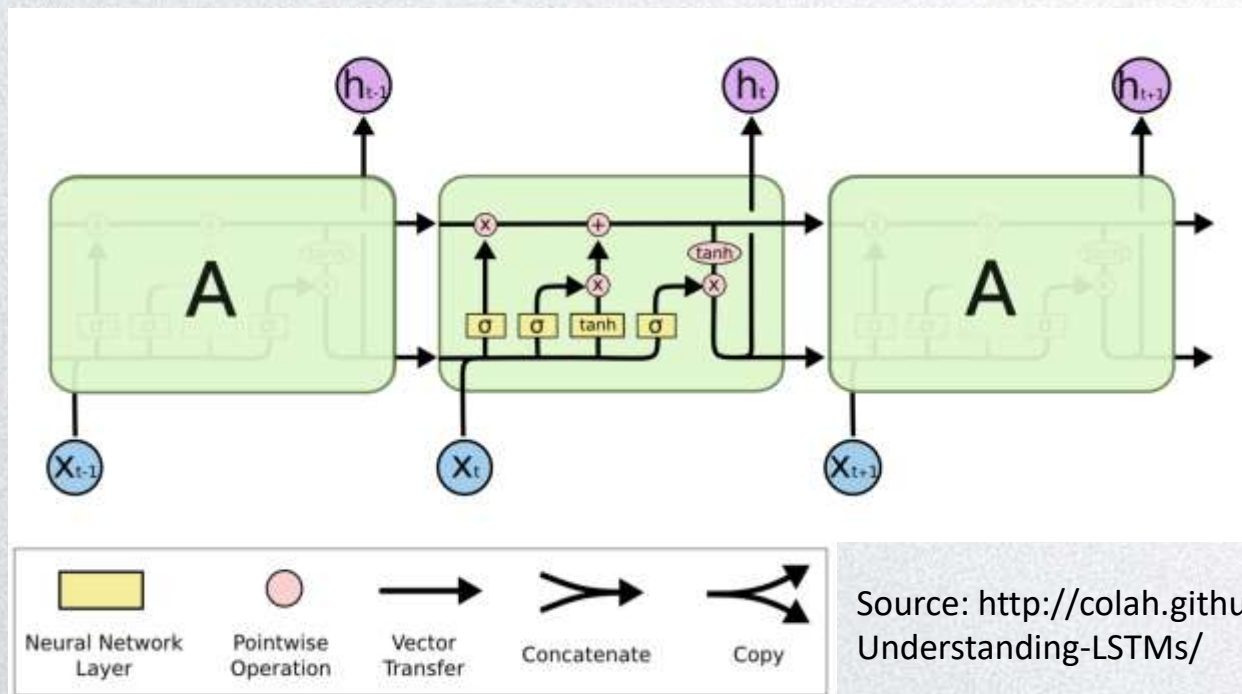
- 遗忘门: $f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$
- 输入门: $i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$
- 输出门: $o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$
- 新单元计算: $c'_t = \tanh(W_c h_{t-1} + U_c x_t + b_c)$
- 根据遗忘门和输入门, 更新长期记忆单元:

$$c_t = f_t \circ c_{t-1} + i_t \circ c'_t$$

- 根据输出门计算隐状态向量: $h_t = o_t \circ \tanh(c_t)$

长短时记忆网络(LSTM)

- LSTM的结构如图所示:



Source: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

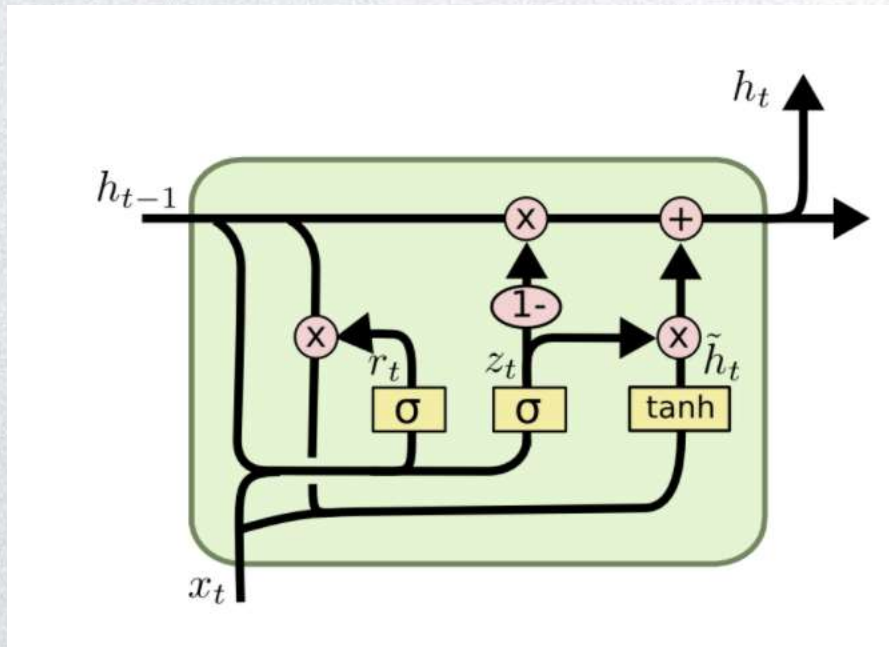


门控循环神经网络 (GRU)

- 只使用隐状态向量 h_t , 但使用更新门和重置门控制 h_t 的更新
 - 更新门: $u_t = \sigma(W_u h_{t-1} + U_u x_t + b_u)$
 - 重置门: $r_t = \sigma(W_r h_{t-1} + U_r x_t + b_r)$
 - 计算新的隐状态向量:
$$h'_t = \tanh(W_h (r_t \circ h_{t-1}) + U_h x_t + b_h)$$
 - 更新隐状态向量: $h_t = (1 - u_t) \circ h_{t-1} + u_t \circ h'_t$

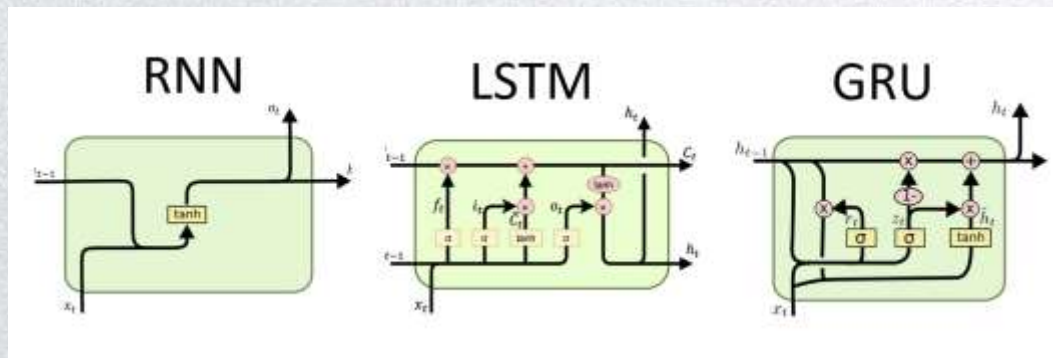
门控循环神经网络 (GRU)

- GRU结构如图所示:



LSTM vs GRU

- 两种被广泛使用的RNN结构
- 通常情况下，可以先尝试使用LSTM
- 如果需要提升训练/推理速度或者控制参数数量，可以切换使用GRU





● 思考：为什么LSTM能缓解梯度消失问题？

- 提示：推导LSTM的 $\frac{\partial c^{(i+1)}}{\partial c^{(i)}}$ ，与RNN中 $\frac{\partial h^{(i+1)}}{\partial h^{(i)}}$ 的形式进行比较